

#ASLI ENGINEERING

Internals of Approximated LRU

SWIPE

BY

ARPIT BHAYANI

Internals of approximated LRU

Redis does not use exact LRU for eviction
because it is space inefficient

24 LRU bits

Hence, it approximates the choice, efficiently

Naive implementation

1. store 'time' (last accessed at) for each object
2. keep them ordered by LAT and keep evicting efficiently
   ↳ Doubly linked list is commonly used for this

Disadvantages

1. DLL equires extra memory (next/prev pointers & key ptr)
2. Whenever LAT changes, it requires a lot of shuffling
   ↳ reduces DB throughput
3. Storing LAT itself takes up 32bits per object

Overall the naive approach is extremely inefficient for Redis

Hence Redis approximates....

ARPIT BHAYANI

Key Decision 1 : Store last Access Time in 24 bits (not 32 bits)

This would help us save 8 bit per object

and for an in-mem DB this is huge saving

But how would we store time then?

last_accessed_at = time.now() & 0x00FFFFFF

seconds

We will just be using 24 LBBs

We are losing on exact time but that is okay

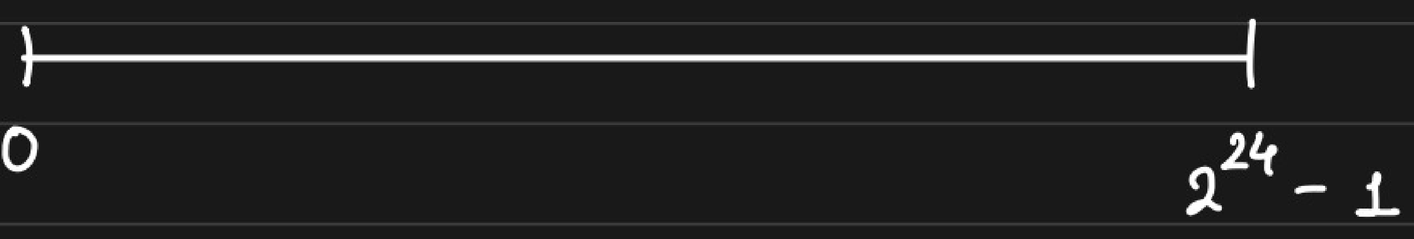
24 bits will help us cover a span of 194 days

Key Decision 2 : Idle time

LRU is all about evicting the key that is least recently used

ie the key that has the most idle time

Our clock is 24 bits hence it repeats



every object will have

time set in this range

ARPIT BHAYANI